

Routing

one model call, then return

```
def take_turn(self, text):
    completion = self.provider.complete(
        system=SYSTEM_PROMPT,
        messages=self.conversation.user_says(text),
        tools=self.schemas(),
    )
    trace.iterations = 1

    if not completion.tool_calls:
        return TurnResult(completion.text, ...)

    call = completion.tool_calls[0]
    result = self.run_action(call)
    dropped = completion.tool_calls[1:]

    return TurnResult(result.text, ...)
```

Deciding

a loop, and one line that feeds it

```
def take_turn(self, text):
    messages = self.conversation.user_says(text)

    for iteration in range(1, MAX_ITERATIONS + 1):
        completion = self.provider.complete(
            system=SYSTEM_PROMPT, messages=messages,
            tools=self.schemas())

        if not completion.tool_calls:
            reply = completion.text
            break

        messages = messages + [assistant(...)]
        for call in completion.tool_calls:
            result = self.run_action(call)
            messages = messages + [tool_result(result)]

    return TurnResult(reply, ...)
```